Arrow - a left-to-right programming language

CS-4031 Compiler Construction, Fall 2026 - Project Phase 1 (interpreter) and Phase 2 (AOT compiler)
Team: Usman, Hamza Hussain, Bahader Ali

1. Overview

Arrow is a small general-purpose language built on one rule: **data always flows left to right.** A value is computed on the left and flows into a variable or an array slot on the right with the arrow `=>`. It is the only way to assign. The language has integer variables, arrays, loops, if/else, and recursive functions, and it is powerful enough to write Insertion Sort and Merge Sort (both are in `examples/`).

```
fn main() {
  6 * 7 => answer;          -- the value 42 flows into 'answer'
  answer => show;           -- ... and flows into 'show' (prints it)
}
```

The project has two deliverables, built from the same lexer and grammar:

Phase	Program	What it does	Tools used
1 (60%)	`./arrow-interp prog.arw`	Interpreter. Runs the program directly. No intermediate code and no LLVM.	Flex, Bison, C
2 (40%)	`./arrowc prog.arw -o prog`	Ahead-of-time compiler. Generates LLVM IR, optimizes it, produces a native executable.	Flex, Bison, C, LLVM (C API)

Everything is written in **C** (Flex and Bison generate C; LLVM is used through its C API). The project runs on **Linux**.

2. Requirements and installation

Tested on Ubuntu 24.04 LTS with GCC 13.3, GNU Make 4.3, Flex 2.6.4, Bison 3.8.2 and LLVM 18.1.3.

Needed for	Package (Debian / Ubuntu)
C compiler, make	`build-essential`
Lexer generator	`flex`
Parser generator	`bison`
Phase 2 only: LLVM headers and libraries	`llvm-18-dev`

Install everything with one command:

```
sudo apt update
sudo apt install build-essential flex bison llvm-18-dev
```

On Fedora the equivalent is `sudo dnf install gcc make flex bison llvm-devel` (not tested). Phase 1 needs only the first three rows of the table. Only LLVM 18 was tested. The code uses only long-standing parts of the LLVM-C API, so neighbouring versions may work, but that is not verified.

3. Building

Open a terminal in the project directory (the one that contains `Makefile`) and run:

```
make
```

This produces two programs in the project directory:

- `arrow-interp` - the Phase 1 interpreter
- `arrowc` - the Phase 2 compiler (also builds the runtime library `build/arrow_rt.o`)

Other targets: `make interp` (interpreter only, no LLVM needed), `make compiler`, `make test` (run the test-suite), `make clean`.

`make` finds `llvm-config` automatically (`llvm-config`, `llvm-config-18`, ...). If it is somewhere else, run `make LLVM_CONFIG=/path/to/llvm-config`.

4. Running Phase 1 - the interpreter

```
./arrow-interp [options] program.arw
```

Option	Meaning
(none)	check the program, then run it
`--check`	parse and check the program only; prints `OK`
`--ast`	print the syntax tree and exit
`-h`, `--help`	show usage

Numbers read by `read()` come from standard input. Exit status: 0 on success, 1 if the program has compile-time errors or fails at run time, 2 for a bad command line.

Examples:

```
$ ./arrow-interp examples/insertion_sort.arw
1
2
5
7
9

$ ./arrow-interp examples/sort_input.arw < examples/numbers.txt     # reads 8 numbers from a file
-45
-2
0
7
7
19
34
100

$ printf '5\n3 1 2 9 -4\n' | ./arrow-interp examples/sort_input.arw    # or type the input through a pipe
-4
1
2
3
9
```

Errors are reported with file and line, and the program is not run:

```
$ ./arrow-interp bad.arw
bad.arw:3: error: variable 'x' holds integer, so an array value cannot flow into it
bad.arw:4: error: variable 'y' is used before it is assigned (use  value => y;  first)
2 error(s); the program was not run.
```

Run-time errors stop the program with exit status 1:

```
arrow: runtime error (line 3): index 3 out of range for array of length 3
```

5. Running Phase 2 - the compiler

```
./arrowc [options] program.arw
```

By default this writes the executable `program` (the file name without `.arw`) into the current directory.

Option	Meaning
`-o FILE`	name of the executable
`-O0` `-O1` `-O2` `-O3`	LLVM optimization level (default `-O2`)
`--emit-ir`	write the LLVM IR exactly as generated (before LLVM optimizes it) to `FILE.ll` and stop

Option	Meaning
`-S`	write the final LLVM IR (after optimization) to `FILE.ll` and stop
`--no-bce`	turn off our bounds-check elimination pass (section 8)
`--stats`	print how many array bounds checks were removed
`-h`, `--help`	show usage

Examples:

```
$ ./arrowc examples/merge_sort.arw -o merge_sort
built merge_sort
$ ./merge_sort
-5
0
3
...

$ ./arrowc --stats examples/bench.arw -o bench
bounds checks: 3 array accesses, 3 checks eliminated, 0 kept
built bench

$ ./arrowc -S -O0 examples/hello.arw -o hello        # look at the generated LLVM IR
wrote hello.ll (IR after -O0)
```

Programs that fail to type-check are rejected with the same messages as the interpreter, no executable is produced, and the exit status is 1. A compiled program behaves exactly like the interpreted one, including run-time error messages and exit status.

`arrowc` finds the runtime library `build/arrow_rt.o` next to itself, so run it from the project directory after `make` (or set the environment variable `ARROW_RT` to the path of `arrow_rt.o`). The linking step needs the system C compiler `cc`.

6. The Arrow language

6.1 A first look

```
fn insertion(a[], n) {                -- a[] is an array parameter
  loop i in 1..n {                    -- i = 1, 2, ..., n-1
    a[i] => key;                      -- value flows into a variable
    i - 1 => j;
    loop j >= 0 & a[j] > key {        -- while loop; & short-circuits
      a[j] => a[j + 1];               -- value flows into an array slot
      j - 1 => j;
    }
    key => a[j + 1];
  }
}

fn main() {
  [5, 2, 9, 1, 7] => data;
  insertion(data, len(data));
  loop k in 0..len(data) { data[k] => show; }
}
```

6.2 Reference

Feature	Syntax and meaning
Program	A list of functions. Execution starts at `main()`, which takes no parameters. Functions may call each other in any order.
Comments	`--` to the end of the line. Write `- -3`, not `--3`, for a double minus.
Types	**Integer**: signed 64-bit, arithmetic wraps around. **Array**: a one-dimensional array of integers.
Variables	Created by their first assignment, visible in the whole function. A variable keeps the type of its first assignment. Never-assigned integers are 0 and never-assigned arrays are empty.
Assign	`expr => x;` and `expr => a[i];` (the value is computed first, then the index)
Print	`expr => show;` prints an integer and a newline
Input	`read()` reads the next integer from standard input
Arrays	`[1, 2, 3]` literal, `new[n]` (n zeros), `len(a)`, `a[i]` (0-based). Arrays are passed and assigned **by reference**.
Functions	`fn name(x, a[]) { ... }`. Parameters are integers, or arrays when written `a[]`. `give expr;` returns an integer immediately; falling off the end returns 0. A call is an expression, or a statement: `f(1, a);`. Recursion works.
If / else	`when cond { ... }`, optionally `otherwise { ... }` or `otherwise when cond { ... }`. Braces are mandatory.
While loop	`loop cond { ... }`
Range loop	`loop i in lo..hi { ... }` runs with i = lo, lo+1, ..., hi-1. `lo` and `hi` are evaluated once. `i` cannot be assigned inside its own loop.
Operators	`+ - * / %`, comparisons `< <= > >= == !=`, logic `&` (and), `|` (or), ~ (not), unary `-`. Division truncates toward zero.
Truth values	0 is false, anything else is true; comparisons and logic give 0 or 1. `&` and `|` do not evaluate the right side when the left side decides the result.

Operator precedence, from loosest to tightest binding: `|`, `&`, `== !=`, `< <= > >=`, `+ -`, `* / %`, unary `- ~`.

6.3 Errors

Checked **before** the program runs (both tools report these with file and line): undeclared variables, assigning an array to an integer variable or the reverse, indexing something that is not an array, wrong number or type of function arguments, calls to undefined functions, duplicate functions or parameters, missing `main`, `main` with parameters, assigning to a loop variable inside its loop, using an array where an integer is needed (operators, conditions, `show`, `len`), integer literals that do not fit in 64 bits, and all syntax errors (the parser recovers at the next `;` and reports several errors at once).

Detected **while the program runs** (exit status 1): array index out of range (including negative), `new` with a negative size, division or remainder by zero, `read()` at the end of input, and - in the interpreter only - recursion deeper than 100000 calls.

7. How it works

7.1 Phase 1: the interpreter (`src/phase1/interp.y`)

```
source -> Flex tokens -> Bison LALR(1) parser -> syntax tree -> semantic checks -> run
```

- **Flex** (`src/lexer.l`) turns the text into tokens (integers, identifiers, keywords, `=>`, `..`, operators) and tracks line numbers.
- **Bison** parses the tokens. The grammar has no conflicts (`%expect 0`). Each rule's C action builds a node of the syntax tree.
- **Semantic analysis** (`src/common/sema.c`) uses hash-table symbol tables (one per function), gives every variable a slot number, and checks types and calls.
- **Execution** is C code in the same Bison file: `ev_int`, `ev_arr`, `exec_stmt` walk the tree and perform each operation directly. No intermediate code is generated and LLVM is not used (the executable is not linked with

it).

7.2 Phase 2: the AOT compiler (`src/phase2/compiler.y`)

```
source -> Flex -> Bison -> syntax tree -> semantic checks -> bounds-check elimination
       -> LLVM IR -> LLVM optimizer -> object file -> linked with C runtime -> executable
```

The grammar and semantic analysis are identical to Phase 1. Instead of executing the tree, the C code in `compiler.y` translates it into LLVM IR with the LLVM-C API:

Arrow	LLVM IR
integer	`i64`
array	`ptr` to a block of `i64`; word 0 is the length, the elements follow
variable	a stack slot (`alloca`); LLVM's own passes turn it into SSA registers
`fn name`	`i64 @f_name(...)`; array parameters are `ptr`. A C `main` calls `f_main`.
`loop cond`, `loop i in a..b`, `when`	basic blocks and conditional branches
`&`, `|`	conditional branches and a `phi` (so the right side is skipped when it should be)
`a[i]`	unsigned compare `i >= len`, branch to a `noreturn` call to the runtime's error routine, otherwise `getelementptr` and `load`/`store`
`give`	`ret`; the rest of the block is dead code in a fresh block
`show`, `read`, `new`, `/` `%` by a non-constant	calls into the small C runtime `src/runtime/arrow_rt.c`

The module is verified (`LLVMVerifyModule`), optimized with LLVM's standard pipeline (`default<O2>` through `LLVMRunPasses`), emitted as an object file for the host machine, and linked with the runtime by `cc`.

7.3 Our optimization: bounds-check elimination (`src/phase2/bce.c`)

Safe array access costs a compare and a branch per access. A pass over the syntax tree (before IR generation) removes the check for `a[i + c]` inside `loop i in LO..HI { ... }` when it can prove the index is in range:

- `LO` is a constant and `LO + c >= 0`, and
- `HI` is `len(a)` or `len(a) - k` with constant `k >= 0` and `c <= k`, and
- the loop body never assigns a new array to `a`. (An array's length never changes, and `i` cannot be modified in the loop, so `i` always stays in `[LO, HI)`.)

The code generator also marks the load of an array's length as invariant (`!invariant.load`), which lets LLVM move it out of loops.

Measured on `examples/bench.arw` (800 million array reads), best of 5 runs on one machine:

Configuration	Time
`-O0`, all checks kept	1.51 s
`-O0` with our pass	0.59 s (2.6 times faster)
`-O2`, LLVM only (`--no-bce`)	about 0.18 s
`-O2` with our pass	about 0.17 s (5 to 10 percent faster, close to the noise)
interpreter (Phase 1)	14.0 s

At `-O2` LLVM already removes most of these checks by itself; our pass also removes the one check LLVM leaves in this program. The pass pays off most with `-O0` and in cases LLVM cannot see through. It is deliberately simple: it only recognizes the patterns above, and the checks in Insertion Sort's inner loop (`a[j]` guarded by `j >= 0`) are kept.

8. Testing

```
make test                        # 133 checks, about 5 seconds
python3 tests/fuzz.py 250 1      # optional: 250 random programs, seed 1
```

`tests/run_tests.sh` runs:

- the 16 programs in `tests/programs/` through the interpreter and through the compiler (twice: `-O2` and `-O0 --no-bce`), comparing the output with the `.expected` files. The expected outputs were computed independently in Python, not copied from our own tools.
- 24 programs in `tests/compile_errors/` that both tools must reject with a specific message
- 16 programs in `tests/runtime_errors/` that must fail with a specific message and exit status 1 (interpreter and compiled program)
- command-line behaviour and the bounds-check statistics

`tests/fuzz.py` generates random valid Arrow programs (loops, arrays, calls, division by zero, out-of-range indexes, rebinding arrays inside loops) and checks that the interpreter, the optimizing compiler and the unoptimized compiler agree on output, exit status and error message. Seeds 1 to 4 compared 873 programs with 0 mismatches; the pass removed 328 of the 15,609 bounds checks in them (about 2 percent), so it is exercised on real data and every removal was confirmed against the unoptimized build.

9. Project files

```
Makefile                build rules (make, make test, make clean)
README.md / README.pdf  this document
src/
  lexer.l               Flex scanner, shared by both phases
  common/
    ast.h, ast.c        syntax-tree node type, list helpers, error reporting
    sema.h, sema.c      semantic analysis: symbol tables, type checks, call resolution
  phase1/
    interp.y            PHASE 1: Bison grammar + tree-walking interpreter + main()
  phase2/
    compiler.y          PHASE 2: Bison grammar + LLVM IR generator + optimizer/linker driver + main()
    bce.h, bce.c        bounds-check elimination pass
  runtime/
    arrow_rt.c          C runtime for compiled programs (print, read, new, error exits)
examples/
  hello.arw             smallest program
  insertion_sort.arw    Insertion Sort
  merge_sort.arw        Merge Sort
  primes.arw            primes below 50
  factorial.arw         recursion
  sort_input.arw        reads numbers from stdin (use with numbers.txt)
  numbers.txt           sample input for sort_input.arw
  bench.arw             benchmark for the bounds-check pass
tests/
  run_tests.sh          test runner (make test)
  fuzz.py               random differential tester
  programs/             programs with .expected output (and .in input where needed)
  compile_errors/       programs that must be rejected; first line says which message
  runtime_errors/       programs that must fail at run time; first line says which message
```

Generated by `make` (removed by `make clean`): `arrow-interp`, `arrowc`, and the directory `build/` with the Flex/Bison output (`build/p1`, `build/p2`) and `build/arrow_rt.o`.

10. Troubleshooting and limitations

- `llvm-config not found`: install `llvm-18-dev`, or pass `make LLVM_CONFIG=/path/to/llvm-config`. The interpreter alone (`make interp`) does not need LLVM.
- `flex: command not found` or `bison: command not found`: install them (section 2).
- `runtime library build/arrow_rt.o not found`: run `make` and call `./arrowc` from the project directory, or set `ARROW_RT`.
- Integers are the only scalar type (no strings, floats or characters); arrays are one-dimensional arrays of integers; functions return an integer.
- Arrays are never freed (no garbage collector); this is fine for short programs.
- A compiled program has no recursion limit: infinite recursion ends with the operating system's stack-overflow crash. The interpreter stops with an error after 100000 nested calls.
- Only Linux on x86-64 was tested. The compiler targets the machine it runs on.